LineTR: A Comprehensive Architecture Guide for Implementation

Implementation Reference Document

Abstract

This document provides a detailed, granular breakdown of the **LineTR** architecture, designed for text line segmentation of challenging historical and palm leaf manuscripts. The goal of this guide is to equip developers and researchers with the necessary architectural details, mathematical formulations, and step-by-step pipeline concepts to build and train the LineTR model from scratch.

Contents

1 Introduction and Core Philosophy

LineTR aims to solve the problem of text line segmentation across diverse datasets without requiring dataset-specific hyperparameters or downsampling entire high-resolution manuscripts, which destroys critical text context.

Unlike traditional approaches that predict pixel blobs (segmentation) or use fixed-size image patches, LineTR treats text-line detection as a geometric curve prediction problem and processes manuscripts via a **two-stage pipeline** utilizing **context-adaptive patches**.

2 System Overview: The Two-Stage Pipeline

The pipeline is split into two major stages:

1. **Stage-1 (Deep Learning Patch Processing):**
   - Computes context-adaptive patch sizes.
   - Processes patches through a shared Vision Transformer (ViT) backbone.
   - Outputs parametric line scribbles via a DETR-style network.
   - Outputs a continuous Text-Energy map via a hybrid CNN-Transformer network.
   - Stitches patch-level outputs into global document-level maps.
2. **Stage-2 (Seam Generation):**
   - Uses the global scribbles and global Text-Energy map to generate tight-fitting polygons around text lines using a Seam Carving algorithm.

—

3 Stage-1: Context-Adaptive Patching

To process documents of vastly different resolutions, fixed-size patches fail by capturing either too much or too little context.

3.1 Adaptive Patch Calculation

1. Sample initial candidate patches of sizes $m \in \{64, 128, 256, 384, 512\}$.
2. Resize these patches to the model's fixed input size $s = 256 \times 256$.
3. Pass through the model to predict lines and calculate the median interline gap δ_{patch}.
4. Scale δ_{patch} back to the original document size: $\delta_{doc} = \delta_{patch} \cdot \frac{m}{s}$.
5. Average the δ_{doc} values across patches to get δ_{avg}.
6. Calculate final patch size $t = \delta_{avg} \times \zeta$, where ζ is a scaling hyperparameter representing the expected number of text lines per patch.

During training and main inference, overlapping patches of size $t \times t$ are sampled and resized to $256 \times 256 \times 3$ to feed into the model networks.

—

4 Stage-1: Deep Neural Networks Architecture

4.1 Shared Encoder

The input to the network is a patch of dimension $H_0 \times W_0 \times 3$ (where $H_0 = 256, W_0 = 256$).

- **Backbone:** Vision Transformer (ViT).
- **Patch Size:** 16×16 pixels.
- **Architecture:** 8 Encoder layers, learnable position embeddings.
- **Output:** Shared feature map $M \in \mathbb{R}^{H \times W \times C}$ where $H = 16, W = 16$, and embedding dimension $C = 256$.

4.2 Line-Parameter Generator (DETR-style)

Instead of predicting pixel boundaries, LineTR predicts strike-through scribbles as parametric lines. A line is represented by three normalized parameters: (μ_x, μ_y, m), where $(\mu_x, \mu_y) \in [0,1]$ is the midpoint and m is the slope.

4.2.1 Architecture

- **Line Queries:** $N = 200$ learnable embeddings.
- **Line Priors:** N horizontal lines uniformly distributed. Prior $A^{(i)}$ initialized as $(\mu_x = 0.5, \mu_y = i/N, m = 0)$.
- **Positional Queries ($Q_p^{(i)}$):** Transformed encodings of Line Priors, added to the input of attention layers.
- **Transformer Decoder:** $L = 8$ layers.
  - *Self-Attention:* Applied among the Line Queries.
  - *Cross-Attention:* Decoupled row-column-based attention between Line Queries and encoder feature map M (enables faster convergence).
- **Prediction Heads:** Output embeddings $E^{(i)}$ are passed through lightweight Feed-Forward Networks (FFN) to yield μ_x, μ_y, m and a confidence probability p.

4.2.2 Loss and Optimization (Bipartite Matching)

Matching ground truth lines to predictions is done via the Hungarian Algorithm. Let l be ground truth and $\hat{l}$ be prediction. The vertical distances between the lines at the left and right patch boundaries are d_{left} and d_{right}.

Geometric Loss:

$$L_{geom}(\hat{l}, l) = d_{left}^2 + d_{right}^2 \tag{1}$$

Matching Cost:

$$C_{match}(\hat{l}, l) = \lambda_{geom} \cdot \frac{L_{geom}(\hat{l}, l)}{\delta} - \lambda_p \cdot p \tag{2}$$

where δ is the median vertical gap between sorted ground truth lines.

Final Optimization Loss: Comprises Focal Loss (L_{focal}) for classification (line vs empty class ϕ) and Geometric Loss for bounding boxes.

$$L_{opt} = \sum_{g(i)=\phi} \lambda_1 L_{focal}(p_i, -) + \sum_{g(i)\neq\phi} \left[\lambda_2 L_{focal}(p_i, +) + \lambda_{geom} \frac{L_{geom}(\hat{l}, l)}{\delta^2} \right] \tag{3}$$

4.3 Text-Energy Map Generator

To aid in building polygons, a continuous map $[0,1]$ representing text likelihood is generated via a Hybrid CNN-Transformer.

4.3.1 Architecture

- **CNN Encoder:** 4 convolutional layers, reducing spatial dimensions by half per layer. Outputs feature map $P \in \mathbb{R}^{16\times16\times256}$.
- **Transformer Decoder:** $R = 3$ layers. Self-attention is applied to ViT map M. Standard cross-attention processes the result with CNN feature P.
- **CNN Decoder:** 4 layers of (convolution + upsampling) equipped with skip connections from the CNN Encoder.
- **Output Layer:** Sigmoid activation yielding map $B \in [0,1]^{256\times256}$.
- **Loss:** Standard Focal loss ($\alpha = 0.25$) against binary ground-truth energy maps.

—

5 Stage-1: Post-Processing

After generating patch-level predictions, they must be merged.

- **Global Scribbles (S):** Patch scribbles are un-normalized and rendered onto the original document dimensions. An Iterative Projection-Merging Algorithm clusters lines based on distance. Each cluster forms a global document text-line scribble.
- **Global Text-Energy Map (B):** Non-overlapping patches are evaluated through the Text-Energy Map generator and stitched together to form a full-document continuous energy map.

—

6 Stage-2: Seam Generation Framework

In the final step, the algorithm builds tight-fitting polygons. LineTR adapts the SeamFormer seam generation method with key modifications:

- **Inputs:** Document-level Scribbles (S) and Global Text-Energy Map (B).
- **No Thresholding:** Map B retains its floating-point values $[0,1]$ instead of converting to harsh binary outputs, preserving weak text signals (like diacritics).

- **No Extra Maps:** LineTR drops reliance on smoothing maps, diacritic maps, or sobel maps because the generated Text-Energy Map B is sufficiently high quality.

A seam carving path-finding algorithm calculates upper, lower, and medial seams surrounding the scribbles navigating through the energy valleys, effectively yielding text-line polygons.

—

7 Training Strategy

Training LineTR is done in two distinct phases:

1. **Phase 1 (60 + 20 epochs):** Train *only* the ViT Backbone and Line-Parameter Generator. Use learning rate 5×10^{-5} for 60 epochs, dropping to 1×10^{-5} for the last 20.
2. **Phase 2 (50 epochs):** Freeze the ViT Backbone and Line-Parameter Generator. Train *only* the Text-Energy Map Generator using learning rate 1×10^{-4}.

The optimizer of choice is AdamW.

8 Summary for Scratch Implementation

If you intend to code this from scratch:

1. Initialize a standard ViT for your backbone.
2. Build a custom Transformer Decoder that implements decoupled row-column cross attention.
3. Write the Hungarian Matching logic using scipy's `linear_sum_assignment`.
4. Build the CNN encoder/decoder with skip connections bridging across a separate 3-layer standard Transformer Decoder.
5. Implement the dynamic patching loop that queries the Line Generator to figure out image density before final extraction.
6. Finally, write a seam carving implementation to connect the generated scribbles and energy map into polygons.